Viability-Guided Evolution of Syntax-Bearing Computational Bodies

Jawaun Brown
2026-06-16

Abstract

Arc 2A asks what interventions make the world's causal grammar visible. Arc 2B asks what computational bodies can express that grammar in the first place. We introduce a typed symbolic architecture-evolution benchmark for syntax-bearing agents. Candidate bodies are motif sets containing components such as tree binders, syntax memory, role-specific heads, world models, intervention planners, counterfactual rollout, formal guards, and shortcut reward heads. Static admissibility rules reject malformed bodies, and viability gates require the final architecture to pass concerned-syntax, self/world, resource, and anti-cheat criteria.

In a 12-seed deterministic design pilot, an 18-cell symbolic Modal sweep, and a learned executable-body validation, reward-only search reaches high apparent return while failing the viability gate. Novelty-only search finds syntax-like bodies, but remains unreliable under the full formal/viability gate. The first executable validation then instantiates four body variants on the Arc 2A learned-agent gate. Only the guarded syntax body passes: parse-high 1.000, action 1.000, low-concern probe rate 0.202, formal validity 1.000, and anti-cheat 0.950. The result is still not a claim that full neural architecture search has been solved. It is a Phase 2B acceptance surface: **accuracy is not architecture, novelty is not viable morphology, and formal validity alone is not concerned syntax.**

1. Why Arc 2B Exists

The maintained-concern arc ended at a representational ceiling. A shared mediated head could learn global h-dependence but did not identify role-specific mediated effects under null-only intervention. That failure has two possible explanations:

1. the agent did not have the right interventions;
2. the agent did not have the right computational body.

Arc 2A takes the first path. Arc 2B takes the second. Its question is not "which model gets the highest reward?" The question is:

> What typed computational morphology can represent causal constituency, intervention invention, and self/world attribution under viability constraints?

This is adjacent to neural architecture search, open-endedness, quality diversity, object-centric world modeling, and neuro-symbolic reasoning. But it is not reducible to any one of them. Generic NAS optimizes architecture performance. Arc 2B optimizes for bodies that pass anti-cheat tests derived from a theory of maintained concern.

2. Architecture Grammar

The pilot uses typed motif sets. Motifs are not implementation classes yet; they are commitments about what distinctions the body can express.

Motif	Meaning
`flat_encoder`	basic observation body
`reward_head`	predicts task return
`shortcut_reward_head`	high train return with shortcut risk
`tree_binder`	can bind parts into constituents
`syntax_memory`	can preserve parse state over time
`world_model`	predicts consequences of actions/interventions
`intervention_planner`	can select information-gathering actions
`role_specific_heads`	can separate role-specific causal components
`counterfactual_rollout`	can simulate intervention alternatives
`formal_guard`	checks admissibility and anti-cheat constraints
`self_repair`	can modify body under guard

3. Static Admissibility

The formal layer rejects bodies that violate dependency rules:

Rule	Reason
`syntax_memory` requires `tree_binder`	syntax memory without bound constituents is ill-typed
`intervention_planner` requires `world_model`	planners need consequence models
`role_specific_heads` require `tree_binder`	role heads attach to constituents
`counterfactual_rollout` requires `world_model` and `intervention_planner`	counterfactuals require model and action
`self_repair` requires `formal_guard`	self-modification needs checks
`shortcut_reward_head` without `formal_guard` is rejected	reward shortcuts are anti-cheat risks
resource cost <= 12	bodies must remain viable

This is a small stand-in for the s(CASP)/ASP + SMT/static-checking layer described in the dynamical-ontology blueprint. The current implementation is plain Python so the gate is auditable and easy to test.

4. Fitness and Gates

Each candidate body receives:

- `train_return`

- `parse_congruity`
- `subtree_facilitation`
- `intervention_invention`
- `self_world_split`
- `anti_cheat`
- `formal_valid`
- `resource_cost`

A body is viable only if all are true:

Gate	Criterion
formal validity	no static violations
resource viability	cost <= 12
parse congruity	>= 0.85
subtree facilitation	>= 0.85
intervention invention	>= 0.55
self/world split	>= 0.75
anti-cheat	>= 0.70
formal guard	present

This explicitly separates reward, novelty, syntax, and admissibility.

5. Search Strategies

The pilot compares three selectors:

Strategy	Description
`accuracy_only`	ranks by train return
`novelty_only`	ranks by descriptor novelty plus a weak syntax term
`viability_guided`	ranks by viability, syntax score, novelty, and resource cost

The viability-guided strategy also repairs dependencies and promotes missing target motifs. This is not meant to be mysterious evolution. It is the first formalized version of the body-side research loop: mutate, check, reject, repair, promote, and archive.

6. Pilot Result

Local command:

```
python3 -m experiments.viable_computational_bodies.search \
  --seeds 12 --generations 18 --population 18 --base-seed 20260616 \
  --out artifacts/viable_computational_bodies/pilot.json \
  --report experiments/viable_computational_bodies/results/pilot_2026_06_16.md
```

Summary:

Strategy	Viable rate	Syntax score	Train return	Formal valid	Best body	Gate
accuracy_only	0.000	0.408	1.000	0.000	shortcut reward body	fail
novelty_only	0.000	0.836	0.480	1.000	counterfactual syntax body	fail
viability_guided	1.000	0.830	0.495	1.000	guarded syntax body	PASS

The full motif strings are recorded in the public result report. The table uses short body labels to keep the manuscript legible.

The diagnostic pattern is the contribution:

1. `accuracy_only` finds high train return, but the body is invalid because it uses a shortcut reward head without a formal guard.
2. `novelty_only` finds a rich body, but it omits the formal guard and fails viability.
3. `viability_guided` finds a lower-train-return body that passes syntax, intervention, self/world, resource, and formal gates.

7. Relation to Current Literature

Mainstream NAS decomposes the problem into search space, search strategy, and performance estimation. That is useful engineering structure, but Arc 2B adds a different acceptance criterion: a body must pass the concerned-syntax and anti-cheat gates. Recent LLM-guided and open-ended NAS systems show that architecture generation can move beyond hand-written cells, but they do not by themselves solve the maintained-concern problem.

Object-centric and graph world models are closer to Arc 2A/2B because they factor the world into interacting components. Recent work on latent state design for world models emphasizes that object-centricity and causality are not identical: an actionable model must represent how interventions propagate through object relations. That is exactly the gap between "there are parts" and "there is concerned syntax."

The pilot also lines up with active causal-discovery work. CausaLab and ACE both treat interventions as budgeted scientific actions. Arc 2B adds the body question: what morphology lets an agent formulate and retain those actions as part of a maintained world?

8. Discovery-Regime Status

This paper adds a second Phase 2 artifact type: **computational body grammar**. Arc 1 had architectures, but they were fixed experimental choices. Arc 2B makes architectures themselves part of the search state and subjects them to formal and viability gates.

The pilot is therefore a discovery-regime transition in methodology, not yet a large empirical discovery about neural networks.

9. Modal Multi-Seed Sweep

The multi-seed sweep was run remotely through Modal:

```
doppler --scope /Users/jawaun/superoptimizers run -- \
  uvx --python 3.12 --from modal modal run \
  experiments/viable_computational_bodies/modal_body_evolution_sweep.py \
  --generations 32 --population 32
```

The sweep used six seeds per strategy, 32 generations, and population 32. Raw JSON remains local under `artifacts/viable_computational_bodies/`; the public report is `experiments/viable_computational_bodies/results/modal_sweep_2026_06_16.md`.

Summary:

Strategy	Viable	Syntax	Train	Formal	Anti-cheat	Cost	Best body	Gate
accuracy_only	0.000	0.417	1.000	0.333	0.400	8.000	guarded shortcut body	fail
novelty_only	0.167	0.835	0.483	1.000	0.725	11.833	guarded syntax body	fail
viability_guided	1.000	0.830	0.495	1.000	0.950	11.000	guarded syntax body	PASS

In the Modal sweep, the accepted body is the same guarded syntax body from the design pilot: tree binding, syntax memory, world modeling, intervention planning, role-specific heads, and a formal guard under the resource budget.

This sharpens the design pilot. `novelty_only` sometimes finds an acceptable body, but it is not reliable enough to pass the strategy-level gate. `viability_guided` passes because it couples syntax, intervention, self/world, formal validity, anti-cheat resistance, and resource viability.

10. Executable Body Validation

The symbolic body grammar is useful only if its motifs correspond to executable mechanisms. The first executable validation maps four bodies onto the learned Arc 2A gate:

Body	Mechanism
`shortcut_reward_body`	flat reward/action head without parse use
`planner_without_tree_body`	learns when to probe but lacks tree-binding features
`restless_tree_body`	parses perfectly but probes every low-concern ambiguity
`guarded_syntax_body`	tree binding + intervention planning + capped calibration guard

Remote command:

```
doppler --scope /Users/jawaun/superoptimizers run -- \
  uvx --python 3.12 --from modal modal run \
  experiments/concerned_syntax/modal_learned_agents_sweep.py \
  --train-trials 3000 --test-trials 1200 --epochs 90
```

Summary:

Body	Parse high	Action	High probe	Low probe	Formal	Anti-cheat	Gate
guarded_syntax_body	1.000	1.000	1.000	0.202	1.000	0.950	PASS
planner_without_tree_body	0.492	0.875	1.000	0.000	0.000	0.700	fail
restless_tree_body	1.000	1.000	1.000	1.000	0.000	0.550	fail
shortcut_reward_body	0.494	0.880	0.000	0.000	1.000	0.400	fail

This gives the symbolic grammar its first behavioral footing. The tree binder is not decorative: without it, intervention observations do not attach to the right constituent. The formal guard is also not decorative: without it, a tree parser becomes restless and passes parse while failing concern.

Next versions should replace these linear executable bodies with richer modules:

- tree binder as a differentiable module or program-induction component
- role-specific heads as mixture-of-experts or routed factor heads
- intervention planner trained on Arc 2A tasks
- formal guard implemented as ASP/s(CASP), Z3, or static Python checks
- quality-diversity archive over syntax, resource, robustness, and proof coverage descriptors

11. Limitations

The current search and executable validation are intentionally small. The executable bodies are linear learned components over vectorized symbolic features, not full neural architectures trained from pixels. The formal guard is implemented as Python logic rather than ASP/s(CASP), Z3, or another external solver. The point is to make the acceptance surface explicit and behaviorally grounded before expensive architecture evolution begins.

The strongest next test is whether executable bodies discovered under this grammar pass the Arc 2A concerned-syntax benchmark without direct access to the hidden parse.

12. Conclusion

Arc 2B reframes architecture search as viability-guided body evolution. The pilot and learned validation show why that matters: train return, novelty, formal validity, tree parsing, and concerned syntax can dissociate. The next phase should not ask for merely "better models." It should ask for bodies whose morphology makes the world's causal grammar learnable without becoming restless or shortcut-driven.

References

Aubin, J.-P. (1991). *Viability Theory*. Birkhauser.

Brown, J. (2026). *The Metric Stack of Concern: From Viability Prediction to Maintained Self/World Boundaries in Minimal Agents*.

Cooper, P., & Velasquez, A. (2026). Active Causal Experimentalist (ACE): Learning intervention strategies via direct preference optimization. arXiv: 2602.02451.

Elsken, T., Metzen, J. H., & Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55), 1-21.

Lehman, J., & Stanley, K. O. (2011). Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2), 189-223.

Kim, K. W. (2026). Latent State Design for World Models under Sufficiency Constraints. arXiv:2605.01694.

Le Lidec, Q., Biza, O., Goudet, O., Balestriero, R., & Lajoie, G. (2026). Causal-JEPA: Learning World Models through Object-Level Latent Interventions. arXiv:2602.11389.

Mouret, J.-B., & Clune, J. (2015). Illuminating search spaces by mapping elites. arXiv:1504.04909.

Revencu, B., Pajot, M., & Dehaene, S. (2026). Representations of geometric shapes have syntactic structure. *Journal of Experimental Psychology: General*, 155(4), 1081-1102. https://doi.org/10.1037/xge0001890

Wu, Y.-F., Lee, M., & Ahn, S. (2024). Neural Language of Thought Models. arXiv:2402.01203.

Yang, J., Zhang, D., Song, X., Dai, Q., Liu, X., Chen, Y., Vashishtha, A., Shi, J., Tan, C., & Peng, H. (2026). CausaLab: A scalable environment for interactive causal discovery toward AI scientists. arXiv:2605.26029.

Zhang, S. et al. (2026). Structuring Open-Ended NAS: Semi-Automated Design Knowledge Structuring with LLMs for Efficient Neural Architecture Search. arXiv:2605.19247.